

DATABASE DESIGN DOCUMENT

MoMo SMS/XML Transaction Processing System

Team: Momo-Flow / Project Team

Nshimiyimana Abdurahim (Rahim)

Keynes Benoit Batsinda

Liana Batsinde (Ange Liana)

Joel Mucyo

1. Introduction and Database Overview

This document presents the database design for the MoMo SMS/XML Transaction Processing System. The database is implemented in MySQL 8.0+ using the InnoDB storage engine. The design uses five main tables: users, categories, transactions, transaction_participants, and system_logs.

The database stores transaction information, classifies transactions, records users involved in transactions, and preserves raw SMS processing information. Primary keys, foreign keys, unique constraints, CHECK constraints, and indexes are used to support data integrity and efficient retrieval.

2. Entity Relationship Diagram (ERD)

The ERD below shows the main entities and their relationships in the database.



3. Design Rationale and Justification

3.1 Database Structure

The database is divided into separate tables for users, categories, transactions, transaction participants, and system logs. This separates different types of information and supports a normalized relational structure.

3.2 Primary Keys

Each main table has a primary key that uniquely identifies its records. The transaction_participants table uses a composite primary key consisting of transaction_id, user_id, and role.

3.3 Foreign Keys and Referential Integrity

Foreign keys connect transactions to users and categories, participants to transactions and users, and system logs to transactions. Referential actions are defined in the SQL schema to maintain consistency when related records are updated or deleted.

3.4 Transaction Participants

The transaction_participants table represents participants associated with a transaction and records their role, such as SENDER, RECEIVER, AGENT, or MERCHANT.

3.5 Data Integrity and Constraints

The schema uses unique constraints for account_number, category_name, and momo_ref_id. CHECK constraints are used for non-negative transaction amounts and fees. ENUM values are used for participant roles and system log processing statuses.

3.6 Data Types

DECIMAL types are used for monetary values. VARCHAR and TEXT are used for identifiers and textual information. DATETIME is used for transaction and creation timestamps. BIGINT is used for transaction and log identifiers where appropriate.

3.7 Indexes

Indexes are defined for transaction dates, user/date combinations, system log status, participant users, and system log transaction references to support common retrieval operations.

3.8 System Logs

The system_logs table preserves raw SMS information and its processing status. transaction_id is nullable because some SMS messages, such as OTP or messages that fail parsing or validation, may not correspond to a successfully processed transaction.

4. Data Dictionary

4.1 Users

Column	Data Type	Key / Constraint	Null	Description
user_id	INT AUTO_INCREMENT	PK	NO	Unique user identifier
full_name	VARCHAR(100)	NOT NULL	NO	Full name
phone_number	VARCHAR(20)		YES	Phone number
account_number	VARCHAR(50)	UNIQUE, NOT NULL	NO	Unique mobile money account identifier

4.2 Categories

Column	Data Type	Key / Constraint	Null	Description
category_id	INT AUTO_INCREMENT	PK	NO	Unique category identifier
category_name	VARCHAR(50)	UNIQUE, NOT NULL	NO	Transaction category name
description	VARCHAR(255)		YES	Description of the category

4.3 Transactions

Column	Data Type	Key / Constraint	Null	Description
transaction_id	BIGINT AUTO_INCREMENT	PK	NO	Internal transaction identifier
momo_ref_id	VARCHAR(50)	UNIQUE, NOT NULL	NO	Original MoMo transaction reference
user_id	INT	FK, NOT NULL	NO	Primary account owner
category_id	INT	FK, NOT NULL	NO	Transaction category
amount	DECIMAL(12,2)	CHECK >= 0	NO	Transaction monetary value
fee	DECIMAL(10,2)	DEFAULT 0, CHECK >= 0	NO	Transaction fee
new_balance	DECIMAL(12,2)	NOT NULL	NO	Balance after transaction
transaction_date	DATETIME	NOT NULL	NO	Date and time of transaction
external_tx_id	VARCHAR(50)		YES	External transaction reference when available

4.4 Transaction Participants

Column	Data Type	Key / Constraint	Null	Description
transaction_id	BIGINT	PK, FK	NO	Referenced transaction
user_id	INT	PK, FK	NO	Referenced user
role	ENUM	PK, NOT NULL	NO	Participant role: SENDER, RECEIVER, AGENT, or MERCHANT

4.5 System Logs

Column	Data Type	Key / Constraint	Null	Description
log_id	BIGINT AUTO_INCREMENT	PK	NO	Unique log identifier
transaction_id	BIGINT	FK	YES	Related transaction when available
raw_sms_date	BIGINT	NOT NULL	NO	Raw SMS timestamp value
sender_address	VARCHAR(50)	NOT NULL	NO	SMS sender address
message_body	TEXT	NOT NULL	NO	Original SMS message body
status	ENUM	NOT NULL	NO	Processing status
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	NO	Log creation timestamp

5. Sample SQL Queries and Results

The following queries demonstrate database functionality through joins, grouping, participant relationships, and system-log retrieval. Screenshots of the query results should be inserted in the indicated locations.

5.1 Transactions with Categories

```
SELECT
    t.transaction_id,
    t.momo_ref_id,
    t.amount,
    t.fee,
    t.transaction_date,
    c.category_name
FROM transactions t
JOIN categories c
    ON t.category_id = c.category_id
ORDER BY t.transaction_date;
```

```
mysql> USE momo_database;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SELECT
->     t.transaction_id,
->     t.momo_ref_id,
->     t.amount,
->     t.fee,
->     t.transaction_date,
->     c.category_name
-> FROM transactions t
-> JOIN categories c
->   ON t.category_id = c.category_id
-> ORDER BY t.transaction_date;
+-----+-----+-----+-----+-----+-----+
| transaction_id | momo_ref_id | amount | fee | transaction_date | category_name |
+-----+-----+-----+-----+-----+-----+
| 1 | 38286062599 | 5000.00 | 100.00 | 2026-05-10 10:15:00 | TRANSFER |
| 2 | 38286062600 | 20000.00 | 0.00 | 2026-05-11 14:30:00 | BANK_DEPOSIT |
| 3 | 38286062601 | 1200.00 | 0.00 | 2026-05-12 09:00:00 | PAYMENT |
| 4 | 38286062602 | 1000.00 | 0.00 | 2026-05-13 18:45:00 | DATA_BUNDLE |
| 5 | 38286062603 | 5000.00 | 0.00 | 2026-05-14 20:10:00 | UTILITY |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.02 sec)
```

5.2 Transactions by User

```
SELECT
    u.user_id,
    u.full_name,
    u.account_number,
    t.transaction_id,
    t.momo_ref_id,
    t.amount,
    t.transaction_date
FROM users u
JOIN transactions t
    ON u.user_id = t.user_id
ORDER BY u.user_id, t.transaction_date;
```

```
mysql> SELECT
->     u.user_id,
->     u.full_name,
->     u.account_number,
->     t.transaction_id,
->     t.momo_ref_id,
->     t.amount,
->     t.transaction_date
-> FROM users u
-> JOIN transactions t
->   ON u.user_id = t.user_id
-> ORDER BY u.user_id, t.transaction_date;
+-----+-----+-----+-----+-----+-----+
| user_id | full_name | account_number | transaction_id | momo_ref_id | amount | transaction_date |
+-----+-----+-----+-----+-----+-----+
| 1 | Eric Manzi | 36521838 | 1 | 38286062599 | 5000.00 | 2026-05-10 10:15:00 |
| 1 | Eric Manzi | 36521838 | 2 | 38286062600 | 20000.00 | 2026-05-11 14:30:00 |
| 1 | Eric Manzi | 36521838 | 5 | 38286062603 | 5000.00 | 2026-05-14 20:10:00 |
| 2 | Divine Uwase | 48291044 | 3 | 38286062601 | 1200.00 | 2026-05-12 09:00:00 |
| 4 | Patrick Mugisha | 84736251 | 4 | 38286062602 | 1000.00 | 2026-05-13 18:45:00 |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.01 sec)
```

5.3 Totals by Category

```
SELECT
    c.category_name,
    COUNT(t.transaction_id) AS transaction_count,
    SUM(t.amount) AS total_amount
FROM categories c
```

```

LEFT JOIN transactions t
  ON c.category_id = t.category_id
GROUP BY c.category_id, c.category_name
ORDER BY total_amount DESC;

```

```

mysql> SELECT
->   c.category_name,
->   COUNT(t.transaction_id) AS transaction_count,
->   SUM(t.amount) AS total_amount
-> FROM categories c
-> LEFT JOIN transactions t
->   ON c.category_id = t.category_id
-> GROUP BY c.category_id, c.category_name
-> ORDER BY total_amount DESC;
+-----+-----+-----+
| category_name | transaction_count | total_amount |
+-----+-----+-----+
| BANK_DEPOSIT | 1 | 20000.00 |
| TRANSFER | 1 | 5000.00 |
| UTILITY | 1 | 5000.00 |
| PAYMENT | 1 | 1200.00 |
| DATA_BUNDLE | 1 | 1000.00 |
+-----+-----+-----+
5 rows in set (0.01 sec)

```

5.4 Participants and Roles

```

SELECT
  t.transaction_id,
  t.momo_ref_id,
  u.full_name,
  tp.role
FROM transaction_participants tp
JOIN users u
  ON tp.user_id = u.user_id
JOIN transactions t
  ON tp.transaction_id = t.transaction_id
ORDER BY t.transaction_id, tp.role;

```

```

mysql> SELECT
->   t.transaction_id,
->   t.momo_ref_id,
->   u.full_name,
->   tp.role
-> FROM transaction_participants tp
-> JOIN users u
->   ON tp.user_id = u.user_id
-> JOIN transactions t
->   ON tp.transaction_id = t.transaction_id
-> ORDER BY t.transaction_id, tp.role;
+-----+-----+-----+-----+
| transaction_id | momo_ref_id | full_name | role |
+-----+-----+-----+-----+
| 1 | 38286062599 | Eric Manzi | SENDER |
| 1 | 38286062599 | Divine Uwase | RECEIVER |
| 2 | 38286062600 | Eric Manzi | RECEIVER |
| 3 | 38286062601 | Divine Uwase | SENDER |
| 3 | 38286062601 | Inyange Retail Store | MERCHANT |
| 4 | 38286062602 | Patrick Mugisha | SENDER |
| 5 | 38286062603 | Eric Manzi | SENDER |
+-----+-----+-----+-----+
7 rows in set (0.01 sec)

```

5.5 System Log Processing Status

```

SELECT
  sl.log_id,
  sl.transaction_id,
  sl.sender_address,

```

```

        sl.status,
        sl.created_at
FROM system_logs sl
ORDER BY sl.log_id;

```

```

mysql> SELECT
->   sl.log_id,
->   sl.transaction_id,
->   sl.sender_address,
->   sl.status,
->   sl.created_at
-> FROM system_logs sl
-> ORDER BY sl.log_id;
+-----+-----+-----+-----+-----+
| log_id | transaction_id | sender_address | status      | created_at      |
+-----+-----+-----+-----+-----+
| 1      | 1              | M-Money       | PROCESSED   | 2026-09-12 17:10:40 |
| 2      | 2              | M-Money       | PROCESSED   | 2026-09-12 17:10:40 |
| 3      | NULL           | M-Money       | OTP_IGNORED | 2026-09-12 17:10:40 |
| 4      | NULL           | M-Money       | FAILED_PARSING | 2026-09-12 17:10:40 |
| 5      | NULL           | M-Money       | FAILED_VALIDATION | 2026-09-12 17:10:40 |
+-----+-----+-----+-----+-----+
5 rows in set (0.01 sec)

```

6. Security, Accuracy, and Unique-Rule Tests

The database was tested using duplicate-value, CHECK-constraint, and foreign-key test cases. Insert screenshots showing the MySQL error/result for each test.

6.1 Duplicate Account Number

```

INSERT INTO users (full_name, phone_number, account_number)
VALUES ('Test Duplicate', '250780000001', '36521838');

```

```

mysql> INSERT INTO users (full_name, phone_number, account_number)
-> VALUES ('Test Duplicate', '250780000001', '36521838');
ERROR 1062 (23000): Duplicate entry '36521838' for key 'users.uq_account_number'

```

6.2 Duplicate MoMo Reference

```

INSERT INTO transactions
(momo_ref_id, user_id, category_id, amount, fee, new_balance, transaction_date,
external_tx_id)
VALUES
('38286062599', 1, 1, 1000.00, 0.00, 39000.00, '2026-05-15 10:00:00', NULL);

```

```

mysql> INSERT INTO transactions
-> (momo_ref_id, user_id, category_id, amount, fee, new_balance, transaction_date, external_tx_id)
-> VALUES
-> ('38286062599', 1, 1, 1000.00, 0.00, 39000.00, '2026-05-15 10:00:00', NULL);
ERROR 1062 (23000): Duplicate entry '38286062599' for key 'transactions.uq_momo_ref_id'

```

6.3 Negative Transaction Amount

```

INSERT INTO transactions
(momo_ref_id, user_id, category_id, amount, fee, new_balance, transaction_date,
external_tx_id)
VALUES
('TEST-NEG-001', 1, 1, -500.00, 0.00, 39000.00, '2026-05-15 11:00:00', NULL);

```

```
mysql> INSERT INTO transactions
-> (momo_ref_id, user_id, category_id, amount, fee, new_balance, transaction_date, external_tx_id)
-> VALUES
-> ('TEST-NEG-001', 1, 1, -500.00, 0.00, 39000.00, '2026-05-15 11:00:00', NULL);
ERROR 3819 (HY000): Check constraint 'chk_non_negative_amount' is violated.
```

6.4 Invalid Foreign Key

```
INSERT INTO transactions
(momo_ref_id, user_id, category_id, amount, fee, new_balance, transaction_date,
external_tx_id)
VALUES
('TEST-FK-001', 9999, 1, 1000.00, 0.00, 39000.00, '2026-05-15 12:00:00', NULL);
```

```
mysql> INSERT INTO transactions
-> (momo_ref_id, user_id, category_id, amount, fee, new_balance, transaction_date, external_tx_id)
-> VALUES
-> ('TEST-FK-001', 9999, 1, 1000.00, 0.00, 39000.00, '2026-05-15 12:00:00', NULL);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`momo_database`.`transactions`, CONSTRAINT `fk_transactions_users` FOREIGN KEY (`user_id`) REFERENCES `users` (`user_id`) ON DELETE RESTRICT ON UPDATE CASCADE)
```

7. CRUD Testing

CRUD testing was performed on the database using INSERT (Create), SELECT (Read), UPDATE, and DELETE operations. The UPDATE test changed the phone number of user_id 1. The DELETE test used a temporary test user so that production sample records were not removed.

7.1 CREATE

Sample records were inserted into the main tables.

```
mysql> INSERT INTO users (full_name, phone_number, account_number)
-> VALUES ('Samuel Habimana', '250781111111', '77123456');
Query OK, 1 row affected (0.03 sec)
```

7.2 READ

SELECT queries were used to retrieve users and other database records.

```
mysql> SELECT *
-> FROM users
-> WHERE account_number = '77123456';
+-----+-----+-----+-----+
| user_id | full_name      | phone_number | account_number |
+-----+-----+-----+-----+
|      11 | Samuel Habimana | 250781111111 | 77123456      |
+-----+-----+-----+-----+
1 row in set (0.01 sec)
```

7.3 UPDATE

The phone number of user_id 1 was updated and then selected to confirm the change.

```
mysql> UPDATE users
-> SET phone_number = '250787777777'
-> WHERE account_number = '77567890';
Query OK, 1 row affected (0.02 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql> SELECT user_id, full_name, phone_number, account_number
-> FROM users
-> WHERE account_number = '77567890';
+-----+-----+-----+-----+
| user_id | full_name          | phone_number | account_number |
+-----+-----+-----+-----+
|      12 | Update Test User  | 250787777777 | 77567890       |
+-----+-----+-----+-----+
1 row in set (0.01 sec)
```

7.4 DELETE

A temporary user was inserted for deletion testing. Insert the final DELETE result and confirmation screenshot here.

```
mysql> DELETE FROM users
-> WHERE account_number = '77123456';
Query OK, 1 row affected (0.03 sec)
```

```
mysql> SELECT *
-> FROM users
-> WHERE account_number = '77123456';
Empty set (0.02 sec)
```

8. SQL-to-JSON Mapping and API Serialization

The JSON representation translates normalized relational records into hierarchical JSON suitable for API responses. Related data is represented using nested objects and arrays.

8.1 Fundamental Data Type Translations

SQL Type	JSON Representation
INT / AUTO INCREMENT	Number (Integer)
BIGINT / AUTO INCREMENT	Number (Integer)
VARCHAR / TEXT	String
DECIMAL(12,2)	Number
DATETIME / TIMESTAMP	ISO 8601 string
ENUM	String
NULL	null

8.2 Relational-to-JSON Mapping

users → account_owner and participant user objects

categories → category

transactions → root transaction attributes

transaction_participants → participants[]

system_logs → system_log

8.3 Complete Transaction JSON

Insert the final team-approved complete transaction JSON example here. It should include the transaction fields and related account owner, category, participants, and system log data.

```
-> WHERE t.transaction_id = 1;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+
| transaction_id | momo_ref_id | amount | fee | new_balance | transaction_date | external_tx_id | owner_user_id | owner_full_name | owner_phn |
| owner_account_number | category_id | category_name | category_description | log_id | raw_sms_date | sender_add |
| message_body |
| status | cre
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+
| 1 | 38286062599 | 5000.00 | 100.00 | 45000.00 | 2026-05-10 10:15:00 | NULL | 1 | Eric Manzi | 250788999 |
| 36521838 | 1 | TRANSFER | Peer-to-peer mobile money transfers between users | 1 | 1715336100000 | M-Money |
| TxId:38286062599 You have transferred 5,000 RWF to Divine Uwase (250791234567) on 2026-05-10 10:15:00. New balance: 45,000 RWF. | PROCESSED | 2026-05-10 17:10:40 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+
1 row in set (0.02 sec)
```

9. Conclusion

The database design provides a relational structure for storing MoMo transaction information, users, categories, participants, and raw SMS processing logs. The design includes primary and foreign keys, unique constraints, CHECK constraints, ENUM values,

